

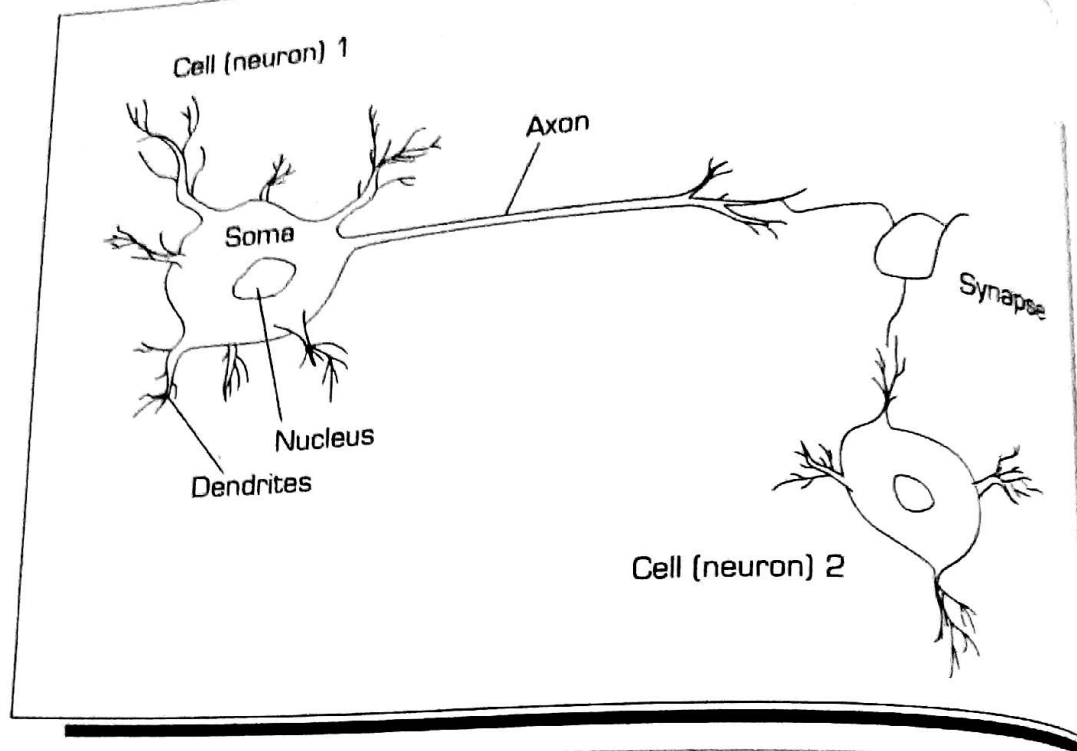


FIGURE 15.1 PORTION OF A NETWORK: TWO INTERCONNECTED BIOLOGICAL CELLS

AI5 IN FOCUS 15.1

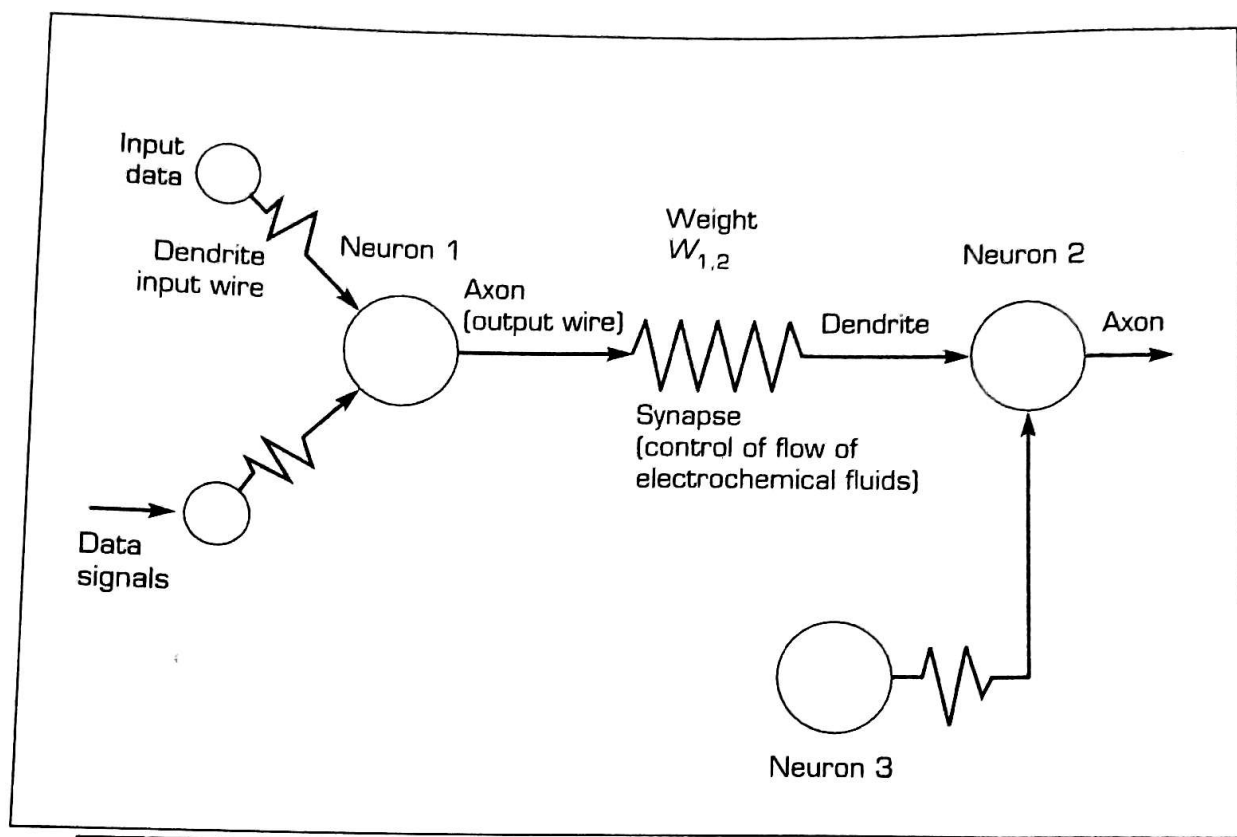
THE RELATIONSHIP BETWEEN BIOLOGICAL AND ARTIFICIAL NEURAL NETWORKS

The list below (Medsker and Liebowitz, 1994, p. 163) shows some of the relationships between biological and artificial networks.

<i>Biological</i>	<i>Artificial</i>
Soma	Node
Dendrites	Input
Axon	Output
Synapse	Weight
Slow speed	Fast speed
Many neurons (10^9)	Few neurons (a dozen to hundreds of thousands)

Zahedi (1993) sees a dual role for artificial neural networks (ANNs). We borrow concepts from the biological world to improve the design of computers. ANN technology is used for complex information processing and machine intelligence. And it can also be used as simple biological models to test hypotheses about biological neuronal information processing.

It is important to recognize that artificial neural networks were originally proposed to model the human brain's activities. The human brain has much more complexity than the model can capture. So, neural computing models are not very accurate representations of real biological systems. Despite extensive research in neurobiology and psychology, important questions remain about how the brain and the mind work. Research and development on ANNs continue to produce interesting and useful results that inspire systems that borrow features from biological systems.

**FIGURE 15.2** THREE INTERCONNECTED ARTIFICIAL NEURONS

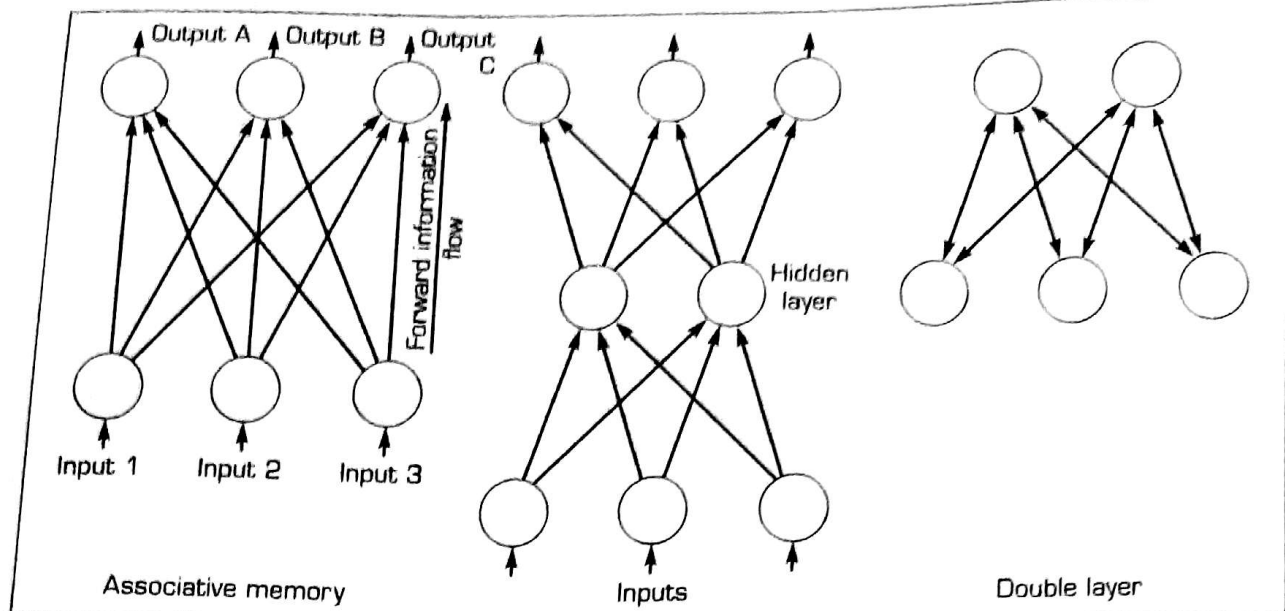


FIGURE 15.8 NEURAL NETWORK STRUCTURES: FEEDFORWARD FLOW

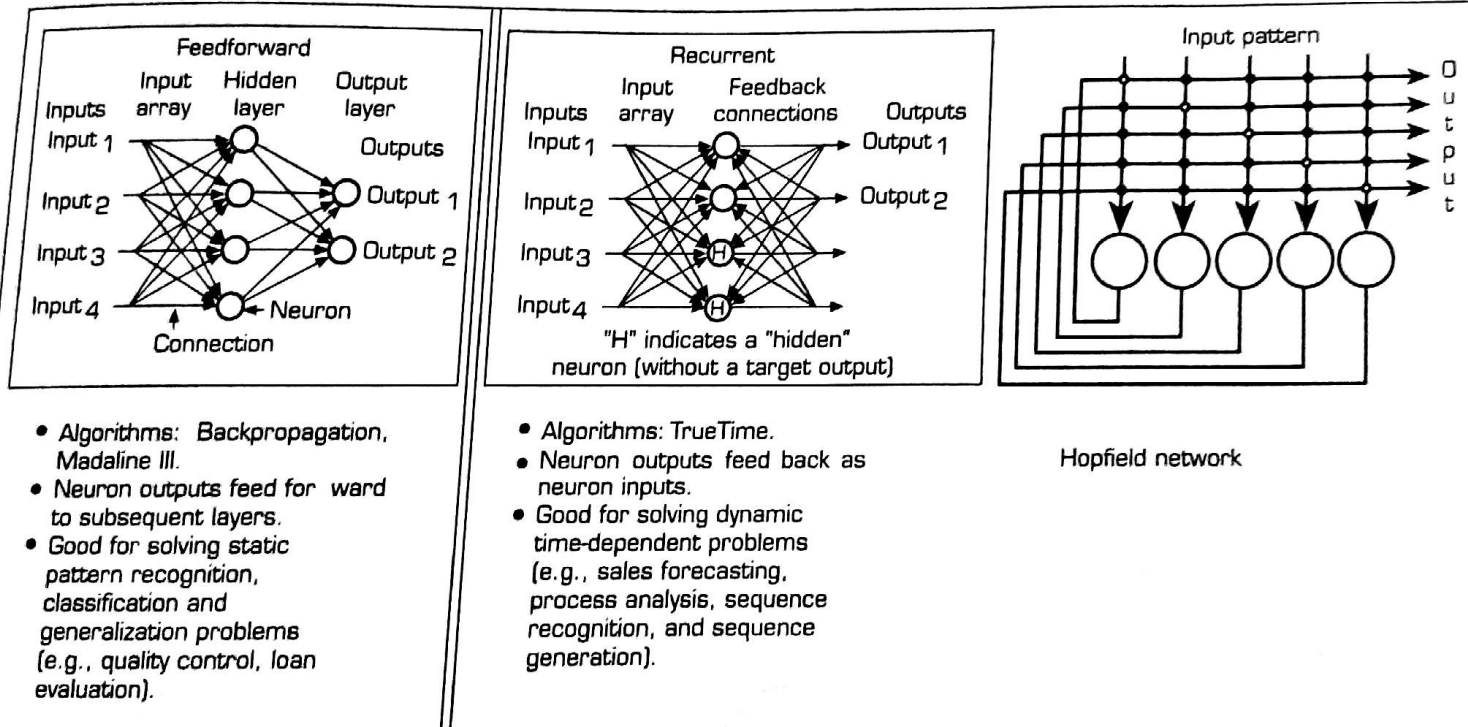


FIGURE 15.9 RECURRENT STRUCTURE COMPARED WITH FEEDFORWARD SOURCE

Source: Based on *PC AI*, May/June 1992, p. 35.

ASSOCIATIVE MEMORY SYSTEMS

Associative memory is the ability to recall complete situations from partial information. These systems correlate input data with information stored in memory. Information can be recalled from even incomplete or *noisy* input. Associative memory systems can detect similarities between new input and stored patterns (see AIS in Focus 15.5) (Haykin 1999; Zurada, 1995).

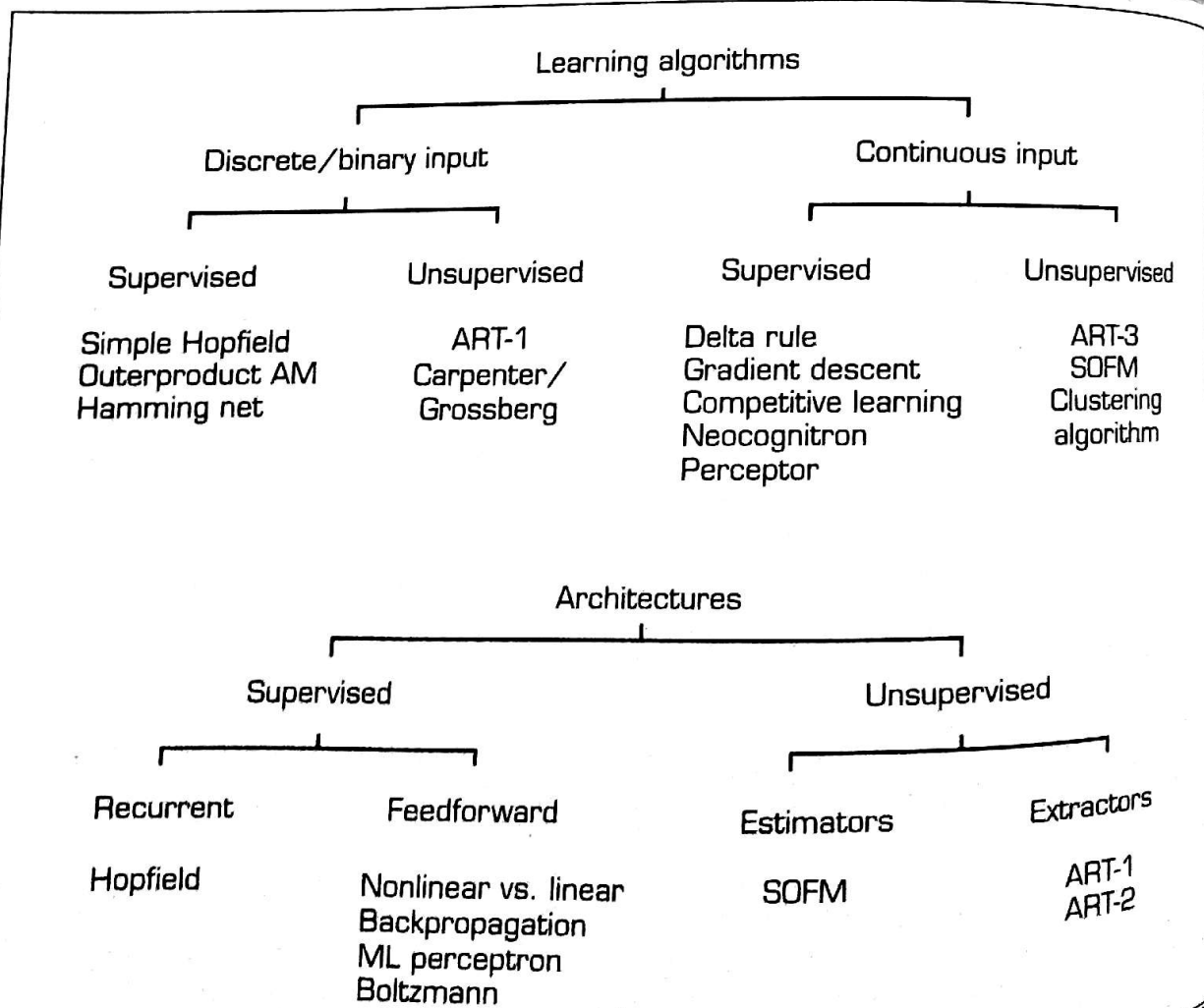
ARNING ALGORITHMS

An important consideration in an ANN is the use of an appropriate **learning algorithm** (or training algorithm). There are hundreds of them. Learning algorithms can be classified as *supervised learning* and *unsupervised learning* (Figure 15.10).

Supervised learning uses a set of inputs for which the appropriate (desired) outputs are known. For example, a historical set of loan applications with the success or failure of the individual to repay the loan has a set of input parameters and presumed known outputs. In one type, the difference between the desired and the actual output is used to calculate corrections to the weights of the neural network. A variation of this approach simply acknowledges for each input trial whether the output is correct as the network adjusts weights in an attempt to achieve correct results. Examples of this type of learning are backpropagation and the Hopfield network.

In **unsupervised learning**, only input stimuli are shown to the network. The network is **self-organizing**; that is, it organizes itself internally so that each hidden processing element responds strategically to a different set of input stimuli (or groups of stimuli). No knowledge is supplied about which classifications (outputs) are correct, and those that the network derives may or may not be meaningful to the network developer (this is useful for cluster analysis).

FIGURE 15.10 TAXONOMY OF ANN ARCHITECTURES AND LEARNING ALGORITHMS



Source: Based on L. Medsker and J. Liebowitz, *Design and Development of Expert Systems and Neural Computing*, New York: Macmillan, 1994, p. 166.

However, setting model parameters can control the number of categories into which the network classifies the inputs. Regardless, a human must examine the final categories to assign meaning and determine the usefulness of the results. Examples of this type of learning are adaptive resonance theory (ART) and Kohonen self-organizing feature maps.

HOW A NETWORK LEARNS

Consider a single neuron that learns the inclusive OR operation—a classic problem in symbolic logic. There are two input elements, X_1 and X_2 . If either or both of them have a positive value, then the result is also positive. This can be shown as

Case	Inputs		Desired Results
	X_1	X_2	
1	0	0	0
2	0	1	1 (positive)
3	1	0	1 (positive)
4	1	1	1 (positive)

The neuron must be trained to recognize the input patterns and classify them to give the corresponding outputs. The procedure is to present to the neuron the sequence of the four input patterns so that the weights are adjusted after each iteration (using feedback of the error found by comparing the estimate to the desired result). This step is repeated until the weights converge to a uniform set of values that allow the neuron to classify each of the four inputs correctly. The results shown in Table 15.1 were produced in Excel. In this simple example a threshold function is used to evaluate the summation of input values. After calculating outputs, a measure of the error (delta) between the output and the desired values is used to update the weights, subsequently reinforcing correct results. At any step in the process for a neuron j we have

$$\text{Delta} = Z_j - Y_j$$

TABLE 15.1 Example of Supervised Learning

Step	X_1	X_2	Z	Initial		Y	Delta	Final	
				W_1	W_2			W_1	W_2
1	0	0	0	0.1	0.3	0	0.0	0.1	0.3
	0	1	1	0.1	0.3	0	1.0	0.1	0.3
	1	0	1	0.1	0.5	0	1.0	0.1	0.5
	1	1	1	0.3	0.5	1	0.0	0.3	0.5
2	0	0	0	0.3	0.5	0	0.0	0.3	0.5
	0	1	1	0.3	0.5	0	0.0	0.3	0.5
	1	0	1	0.3	0.7	0	1.0	0.5	0.7
	1	1	1	0.5	0.7	1	0.0	0.5	0.7
3	0	0	0	0.5	0.7	0	0.0	0.5	0.7
	0	1	1	0.5	0.7	1	0.0	0.5	0.7
	1	0	1	0.5	0.7	0	1.0	0.7	0.7
	1	1	1	0.7	0.7	1	0.0	0.7	0.7
4	0	0	0	0.7	0.7	0	0.0	0.7	0.7
	0	1	1	0.7	0.7	1	0.0	0.7	0.7
	1	0	1	0.7	0.7	1	0.0	0.7	0.7
	1	1	1	0.7	0.7	1	0.0	0.7	0.7

Parameters: alpha = 0.2; threshold = 0.5.

where Z and Y are the desired and actual outputs, respectively. Then, the updated weights are

$$W_i(\text{final}) = W_i(\text{initial}) + \alpha \times \delta \times X_i$$

where α is a parameter that controls how fast the learning takes place.

As shown in Table 15.1, each calculation uses one of the X_1 and X_2 pairs and a corresponding value for the OR operation along with initial values W_1 and W_2 of a neuron's weights. Initially, the weights are assigned random values and the learning rate α , is set low. Delta is used to derive the final weights, which then become the new weights in the next iteration (row).

The initial values of weights for each input are transformed using the above equation to assign values that are used with the next input (row). The threshold value (0.5) sets the output Y to 1 in the next row if the weighted sum of inputs is greater than 0.5; otherwise, Y is set to 0. In the first step, two of the four outputs are incorrect (delta = 1) and a consistent set of weights has not been found. In subsequent steps, the learning algorithm improves the results until finally producing a set of weights that give the correct results ($W_1 = W_2 = 0.7$ in step 4 of Table 15.1). Once determined, a neuron with these weight values can quickly perform the OR operation.

In developing an ANN, an attempt is made to fit the problem characteristic to one of the known learning algorithms. There are software programs for all the different algorithms, but it is best to use a well-known and well-characterized one, such as backpropagation, which we describe next.

5.12 BACKPROPAGATION

Backpropagation (short for *back error propagation*) is the most widely used learning algorithm (Haykin, 1999; Principe et al., 2000). It is a very popular technique that is easy to implement. It does require a set of training data before the network can be used with other data. A backpropagation network includes one or more hidden layers. This type of network is considered *feedforward* because there are no interconnections between the output of a processing element and the input of a node in the same layer or in a preceding layer. Externally provided correct patterns are compared with the neural network output during (supervised) training, and feedback is used to adjust the weights until all the training patterns are as correctly categorized as is possible by the network (the error tolerance is set in advance).

Starting with the output layer, errors between the actual and desired outputs are used to correct the weights for the connections to the previous layer (Figure 15.11). For any output neuron j , the error (δ_j) = $(Z_j - Y_j) (df/dx)$, where Z and Y are the desired and actual outputs, respectively. The sigmoid function, $f = [1 + \exp(-x)]^{-1}$, is an effective way to compute the output of a neuron in practice, where x is proportional to the sum of the weighted inputs to the neuron. With this function, the derivative of the sigmoid function $df/dx = f(1 - f)$ and the error is a simple function of the desired and actual outputs. The factor $f(1 - f)$ is the *logistic function*, which serves to keep the error correction well bounded. The weights of each input to the j th neuron are then changed in proportion to this calculated error. A more complicated expression can be derived to work backward in a similar way from the output neurons through the hidden layers to calculate the corrections to the associated weights of the inner neurons.⁴

⁴This complicated method is an iterative approach to solving a nonlinear optimization problem that is very similar in meaning to the one characterizing multiple linear regression.